Ángeles Araiza García A00574806

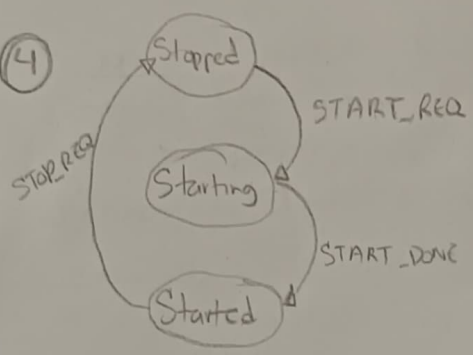
① Verilog es un tipo de programa descriptivo de hardware que te permite implementar código de manera física a través de el lenguaje que utiliza.

② En las asignaciones bloqueantes se ejecuta de manera secuencial las instrucciones, es decir, en cuanto lee la línea se asigna ese valor, en cambio en las no bloqueantes, se ejecuta todo el bloque de manera simultanea. Las asignaciones bloqueantes se usan para la logica combinacional (=), y las no-bloqueantes para la logica secuencial (<=)

Cuanto, cuales?

③ Una variable de tipo reg es una variable de salida que nos puede dar 4 tipos, un 1, 0 lógico, un X como don't care, y Z, como alta impedancia

④




```
module FSM(
    input clk,
    input rst,
    input START_REQ
    input START_DONE
    input STOP_REQ );

reg [1:0] estado_actual;
parameter Stopped=0, Starting=1, Started=2;

always @(posedge clk or posedge rst)
    if(rst) begin
        start_req<=0,
        START_DONE<=0,
        estado_actual<= Stopped,
    else begin
    case(estado_actual)
        STOPPED:
        if(start_req) begin
            estado_actual<= STARTING;
        else
            estado_actual<= STOPPED;
        end
```

3/4

→ Faltó 1 always (actualización)

STOP_REQ<=0;

STAR_DONE<=0
STOP_REQ<=0

```
STARTING:
if(START_DONE) begin
    start_REQ=0 , STOP_REQ<=0
    estado_actual<= STARTED
else
    estado_actual<= estado_actual
end

STARTED: if(STOP_REQ) begin
    estado_actual <= STOPPED;
    START_REQ<=0;
    START_DONE<=0;
end
default: estado_actual<= stopped
endcase
endmodule.
```

⑤

```
module FSM_tb ();
  reg clk;
  reg rst;
  reg START_REQ;
  reg START_DONE;
  reg STOP_REQ;

wire [1:0] estado_actual;

initial
  begin
  clk=0;
  always
    #5 clk=~clk;
end
```

```
initial
  begin
  $display("Simulación iniciada")
  rst = 1;
  #10
  rst = 0;
  #10

  STAR_REQ=1;
  #100;
  START_DONE=1;
  #100;
  STOP_REQ=1;
  #100;
  START_REQ=0;
  #100;
  START_DONE=0;
  #100
  STOP_REQ=0
  $finish
end
endmodule
```

---

⑥ Para realizar un contador ascendente y descendente se necesita de una señal que indique que hay un cambio, por lo que tendremos esa entrada, además del reloj y el rst. Luego necesitamos hacer un registro de un contador con los suficientes bits para llegar a (ejemplo 100) que en este caso son 7 reg [7:0] contador. Una vez teniendo esto, podemos iniciar nuestro bloque con la indicación del cambio de reloj o rst always @(posedge clk, or posedg rst), dentro validamos si se presiono un rst, todo se reiniciana, sino, continuamos con nuestro bloque con la condición de verificar si se presiona la señal de la manera en la que estamos contando, lo que nos darían dos condicionales

```
IF(boton)                    boton = 1 //ascendente
                                   = 0 // descendente
  if(contador==parametro)
    contador <= 0;
  else
    contador <= contador + 1
else
  if(contador == 0)
    contador <= parametro.
  else
    contador <= contador - 1
```

Ángeles Araiza García A00574806

7) Para elaborar este módulo necesitamos el registro de este contador

```
reg [9:0] contador
parameter MAX=1000;
input clk
```

Con estas variables ya podrás inicializar tu bloque del contador donde lo unico que se necesita validar es que el contador sea igual a tu max (1000) si eso pasa, se reinicia el contador, mientras eso no pase, el contador sumara 1 (+1) en cada ciclo

8) Este contador solo necesita de logica combinacional donde se especifica cada caso y estado del contador, es decir, crear un estado para cada numero y a traves de una maquina indicar cual sera el siguiente número para volverlo un ciclo si asi se pide, o seguir con las combinaciones que requiera el conteo no lineal

CRONOMETRO OK